

OpenLcbCLib

Developer Guide

From first project to a fully featured OpenLCB/LCC node

Covers all supported platforms and IDEs

Table of Contents

- 1. Introduction**
- 2. OpenLCB/LCC Concepts**
- 3. Project File Structure**
- 4. The Node Wizard in Detail**
 - 4.1 Node Type and Project Options
 - 4.2 CDI Editor
 - 4.3 FDI Editor (Train Nodes Only)
 - 4.4 Target Platform
 - 4.5 Callbacks
 - 4.6 Generated Files
 - 4.7 Arduino vs. Non-Arduino Differences
- 5. `openlcb_user_config.h` in Depth**
 - 5.1 Feature Flags
 - 5.2 Message Buffer Pool
 - 5.3 Node Count
 - 5.4 Event Counts
 - 5.5 Stream Transport
 - 5.6 Train Protocol
 - 5.7 Listener Alias Verification
 - 5.8 DCC CV Programming Space
- 6. Initialization — `main.ino`**
 - 6.1 The CAN Config Struct
 - 6.2 The OpenLCB Config Struct
 - 6.3 Setup and Loop
- 7. Implementing the Drivers**
 - 7.1 CAN Bus Drivers
 - 7.2 OpenLCB Platform Drivers
- 8. Callbacks — Where Your Application Lives**
- 9. Events in Depth**
- 10. Configuration Memory**
- 11. Train Nodes (Brief Overview)**
- 12. Using PlatformIO**
- 13. Troubleshooting**

1. Introduction

OpenLcbCLib is a C library that implements the OpenLCB/LCC (Layout Command Control) protocol suite for microcontrollers. It is designed to be portable across any hardware platform and IDE, with no dynamic memory allocation — all buffers are statically sized at compile time, making it suitable for processors with very limited RAM.

This guide walks you through creating a complete node project from scratch using the Node Wizard tool, then explains every generated file so you understand exactly what the code does and how to extend it.

What the Library Does (and Does Not Do)

The library handles all OpenLCB/LCC protocol compliance: alias negotiation, node initialization, event transport, datagram handling, SNIP, configuration memory access, stream transport, and train control. It does NOT include any hardware-specific code. You supply short driver functions that read and write your CAN hardware, and the library calls them as needed.

Platform Support

Because the library contains no hardware-specific code, it can be ported to any processor and toolchain. You provide a small set of driver functions for your CAN hardware and platform services, and the library does the rest. The following processors already have driver code written and example projects included in the library:

Processor / Board	Example IDE / Toolchain
ESP32 (CAN via TWAI)	Arduino IDE, PlatformIO
ESP32 (WiFi GridConnect)	PlatformIO
Raspberry Pi Pico (RP2040, MCP2517FD)	Arduino IDE (Earle Philhower core)
STM32F4xx (F407 Discovery)	STM32CubeIDE
Microchip dsPIC33	MPLAB X
TI MSPM0 (MSPM0G3507 LaunchPad)	Code Composer Studio Theia
macOS (GridConnect over TCP)	Xcode, PlatformIO

Porting to a new processor means writing two short driver files (CAN hardware and platform services). The Node Wizard generates TODO-marked stubs for both so you know exactly what functions need to be filled in. Section 7 covers this in detail.

This guide focuses on ESP32 as the worked example. All concepts apply equally to any other target.

2. OpenLCB/LCC Concepts

You do not need to read the full OpenLCB/LCC standards to use this library, but a basic understanding of the following concepts will help you make good design decisions.

The Network

All LCC nodes connect to a shared CAN bus (or a TCP/IP gateway). Every node can hear every message on the bus. Nodes identify themselves with a globally unique 48-bit Node ID.

Events (Producer-Consumer Model)

The most common way nodes communicate is through Events. An event is a 64-bit number that means “something happened.” A node that detects a condition (button pressed, train entered block) fires an event. Any node that cares about that event reacts to it (throws a switch, changes a signal). The producer and consumer do not know about each other — they are loosely coupled through the shared event ID.

Configuration (CDI)

A Typical node exposes configurable settings to the layout operator via CDI (Configuration Description Information). The CDI is an XML document stored in the node that describes what settings are available. JMRI and other tools read the CDI and show a configuration panel.

Node Types and What They Compile

Node Type	Protocols Compiled	Typical Use
Basic	Events only	Simple sensor or output that only fires/reacts to events.
Typical	Events + Datagrams + Config Memory	Most common. Sensor/output with user-configurable event IDs.
Train	Typical + Train + FDI	Locomotive decoder. Responds to speed/function commands.
Train Controller	Typical + Train + Train Search	Throttle or command station. Commands trains.
Bootloader	Datagrams + Config Memory + Firmware	Firmware upgrade image. Minimal protocol footprint.

3. Project File Structure

Understanding the file layout tells you where to look when you need to change something. The files in `src/openlcb/` and `src/drivers/` are library code — do not edit them. The files at the top level are yours.

```
YourProject/          <- Arduino sketch folder
YourProject.ino       <- Main sketch (setup + loop)
openlcb_user_config.h <- REQUIRED: buffer sizes & feature flags
openlcb_user_config.c <- User node parameter data
src/
  application_callbacks/
    callbacks_can.h / .cpp <- CAN observation hooks
    callbacks_olcb.h / .cpp <- Application logic hooks
    callbacks_config_mem.h / .cpp <- Config memory hooks
  application_drivers/
    esp32_can_drivers.h / .cpp <- CAN hardware interface
    esp32_drivers.h / .cpp <- Platform services (NVS, mutex, reboot)
  xml_files/
    cdi.xml           <- CDI definition
    fdi.xml           <- FDI definition (train nodes only)
  openlcb_c_lib/      <- Library core (do not edit)
  openlcb/            <- Protocol engine
    openlcb_config.h/c - Stack init API
    openlcb_types.h    - All data types
    openlcb_defines.h  - Version and constants
    openlcb_node.h/c   - Node allocation and management
    openlcb_application.h/c - Event registration/sending API
    openlcb_application_broadcast_time.h/c
    openlcb_application_train.h/c
    openlcb_application_dcc_detector.h/c
    openlcb_application_dcc_cv.h/c - DCC CV programming space (0xF8)
    openlcb_buffer_fifo.h/c - FIFO buffer management
    openlcb_buffer_list.h/c - List buffer management
    openlcb_buffer_store.h/c - Buffer pool allocation
    openlcb_float16.h/c - Float16 conversion
    openlcb_gridconnect.h/c - GridConnect framing
    openlcb_utilities.h/c - Helper functions
    openlcb_login_statemachine.h/c
    openlcb_login_statemachine_handler.h/c
    openlcb_main_statemachine.h/c
    protocol_event_transport.h/c - Event protocol engine
    protocol_datagram_handler.h/c - Datagram protocol engine
    protocol_config_mem_read_handler.h/c
    protocol_config_mem_write_handler.h/c
    protocol_config_mem_operations_handler.h/c
    protocol_config_mem_stream_handler.h/c
    protocol_stream_handler.h/c - Stream transport engine
    protocol_snip.h/c - Node identification protocol
    protocol_message_network.h/c - Message network handler
    protocol_broadcast_time_handler.h/c
    protocol_train_handler.h/c
    protocol_train_search_handler.h/c
  drivers/canbus/      <- CAN state machines (do not edit)
  drivers/tcp_ip/      <- TCP/IP transport; compiles out when CAN is selected
  utilities/          <- Shared utilities
```

4. The Node Wizard in Detail

The Node Wizard is a browser-based code generator at tools/node_wizard/node_wizard.html. Open it in any modern browser — no installation or internet connection needed. Work through the sidebar sections from top to bottom. All settings persist automatically in your browser's local storage. Use Save Project / Load Project in the header to save and reload a project across browsers or machines.

4.1 Node Type and Project Options

The first step is selecting the role of your node. This determines which feature flags are set in `openlcb_user_config.h`, which sidebar sections are enabled, and which callback stubs are generated.

Node Type	Description
Basic	Sends and receives events only. No configurable settings. CDI and FDI sections are disabled.
Typical	Events plus user-configurable settings stored in config memory. The CDI section is enabled. Most common choice for sensors, turnout drivers, and signal heads.
Train	Locomotive decoder. CDI and FDI sections are both enabled.
Train Controller	Throttle or command station.
Bootloader	Firmware-upgrade image only: datagrams, config memory and firmware upgrade, nothing else. Pairs with the standalone bootloader projects.

Project Options

Below the node type buttons, the Project Options panel is always visible. Fill this in before generating — the values flow into the generated code and the ZIP filename.

Field	Description
Project Name	Short identifier for your project, e.g. <code>my_signal_controller</code> . Used as the ZIP filename when you download. Stick to letters, numbers, and underscores.
Author	Your name or organisation. Written into the generated file headers.
Node ID	Your node's globally unique 48-bit identifier in dotted hex format: <code>xx.xx.xx.xx.xx.xx</code> (e.g. <code>05.01.01.00.00.01</code>). The field auto-formats as you type and shows a live valid / invalid indicator.
Broadcast Time	Optionally compile in the OpenLCB Broadcast Time protocol. Choose None (default), Producer, or Consumer.
Firmware Update	Enables the OpenLCB Firmware Upgrade Protocol so the node can accept a new firmware image over the bus without physical access. Requires config memory.
DCC CV Programming	Serves decoder CVs through memory space <code>0xF8</code> so JMRI DecoderPro can program through this node (command stations and train nodes). Adds the <code>dcc_cv_read</code> / <code>dcc_cv_write</code> callbacks. Requires config memory.

4.2 CDI Editor

The CDI editor lets you describe what settings your node exposes to configuration tools like JMRI. The CDI is an XML document. The editor provides a visual interface so you do not need to write raw XML. This section is enabled for Typical, Train, and Train Controller nodes.

CDI Structure

A CDI document has two main parts: an identification block (manufacturer, model, versions) and one or more configuration segments. A segment is a named block of settings stored at a specific memory address.

CDI Element	Description
<code><string size="N"></code>	Text field N bytes long. Used for names and descriptions.
<code><eventid></code>	Stores a 64-bit event ID (8 bytes). Most common field type.

CDI Element	Description
<int size="N">	Integer value N bytes (1, 2, or 4).
<group replication="N">	Repeats the group N times. Useful for multi-channel nodes.
<segment space="253">	Space 253 is the user configuration space JMRI reads/writes.

Important: The total bytes in your CDI segments must match the actual memory layout in your config_mem_read/write driver functions. Mismatches cause JMRI to read garbage values.

4.3 FDI Editor (Train Nodes Only)

The FDI editor is only enabled for Train nodes. It describes what DCC functions your decoder supports (headlight, bell, horn, etc.). Each function entry maps a function number (F0-F28) to a name and behaviour type (binary on/off, or momentary).

4.4 Target Platform

The Target Platform section is where you select your target hardware. This is one of the most important steps — the platform you choose determines two things: whether the output uses Arduino or non-Arduino layout, and whether the driver files come pre-filled with working code or just TODO stubs.

Platform	Framework Layout	Driver Code
ESP32 + TWAI	Arduino / PlatformIO	Pre-filled
ESP32 + WiFi GridConnect	Arduino / PlatformIO	Pre-filled
RP2040 + MCP2517FD	Arduino (Earle Philhower core)	Pre-filled
STM32F4xx + CAN	STM32 HAL (CubeIDE)	Pre-filled
TI MSPM0 + MCAN	TI DriverLib (CCS / Theia)	Pre-filled
None / Custom	Your own toolchain	TODO stubs

4.5 Callbacks

Callbacks are the hooks between the library and your application. The Callbacks section lets you choose which protocol events your application needs to respond to. Each selected callback generates a stub with a comment explaining what it should do. Unselected callbacks are omitted entirely, keeping the code clean.

4.6 Generated Files

Click the Generated Files tile in the sidebar to preview the output file tree. From there, click Download ZIP to get your project.

Generated File	What It Is
openlcb_user_config.h	Buffer depths and feature flags. Required.
openlcb_user_config.c	The node_parameters struct with your node's metadata.
main.ino (Arduino)	Application entry point — .ino so Arduino IDE recognises it.
main.c (non-Arduino)	Application entry point — plain C for other toolchains.
application_callbacks/callbacks_*.h	Callback header stubs.
callbacks_*.cpp (Arduino)	Callback implementations — .cpp so C++ Arduino APIs are available.
callbacks_*.c (non-Arduino)	Callback implementations — plain C.
application_drivers/openlcb_can_drivers.h	CAN hardware abstraction header.
openlcb_can_drivers.cpp / .c	CAN driver implementation.
application_drivers/openlcb_drivers.h	Platform driver header (lock/unlock, config read/write, reboot).
openlcb_drivers.cpp / .c	Platform driver implementation.
xml_files/cdi.xml	Your CDI XML definition.
xml_files/fdi.xml	Your FDI XML (train nodes only).
GETTING_STARTED.txt	Step-by-step instructions included in every ZIP.
<type>_project.json	Saved Wizard state (e.g. typical_project.json). Load it from the header to resume editing.

4.7 Arduino vs. Non-Arduino Differences

The platform you select in the Target Platform step automatically determines the output layout. There is no manual checkbox — the Wizard sets Arduino mode based on the chosen platform. This changes two things in the generated output:

- **Folder structure** — all driver, callback, library, and XML folders move under src/. Arduino IDE only compiles C/C++ files that are in a subfolder named exactly src/.
- **File extensions** — driver and callback source files are generated as .cpp instead of .c. This lets you call C++ Arduino APIs such as Serial, Wire, and SPI directly inside your driver and callback implementations.

Note: The openlcb_c_lib/ folder is a placeholder — the Wizard creates the empty folder structure but does not copy the library. You must copy the openlcb/, drivers/canbus/, and utilities/ source files there yourself.

5. openlcb_user_config.h in Depth

This file is the single most important file in your project. The library uses these constants at compile time to determine how much memory to allocate. If a value is wrong, the library will either waste RAM or crash at runtime.

5.1 Feature Flags

```
// Uncomment the protocols your node uses.
// Unused protocols are excluded from the build entirely.

#define OPENLCB_COMPILE_EVENTS           // Event producer/consumer
#define OPENLCB_COMPILE_DATAGRAMS       // Required for Config Memory
#define OPENLCB_COMPILE_MEMORY_CONFIGURATION // CDI and settings via JMRI
// #define OPENLCB_COMPILE_FIRMWARE     // Firmware upgrade support
// #define OPENLCB_COMPILE_STREAM        // Stream transport protocol
// #define OPENLCB_COMPILE_BROADCAST_TIME // Clock synchronization
// #define OPENLCB_COMPILE_TRAIN         // Locomotive control
// #define OPENLCB_COMPILE_TRAIN_SEARCH  // Throttle train discovery
// #define OPENLCB_COMPILE_DCC_DETECTOR  // DCC detection protocol
// #define OPENLCB_COMPILE_DCC_CV        // DCC CV programming space 0xF8

// Transport -- exactly one must be defined (see Section 6)
#define OPENLCB_COMPILE_CAN
// #define OPENLCB_COMPILE_TCP
```

Note: OPENLCB_COMPILE_MEMORY_CONFIGURATION requires OPENLCB_COMPILE_DATAGRAMS. OPENLCB_COMPILE_FIRMWARE and OPENLCB_COMPILE_DCC_CV require OPENLCB_COMPILE_MEMORY_CONFIGURATION. OPENLCB_COMPILE_DCC_DETECTOR requires OPENLCB_COMPILE_EVENTS. OPENLCB_COMPILE_BROADCAST_TIME requires OPENLCB_COMPILE_EVENTS. OPENLCB_COMPILE_TRAIN_SEARCH requires both OPENLCB_COMPILE_TRAIN and OPENLCB_COMPILE_EVENTS.

There is also a convenience preset: defining OPENLCB_COMPILE_BOOTLOADER automatically enables DATAGRAMS, MEMORY_CONFIGURATION, and FIRMWARE.

5.2 Message Buffer Pool

The library uses a pool of fixed-size message buffers. Choose depths that balance RAM use with the traffic your node will handle.

Buffer Type (define name)	Size per Buffer
USER_DEFINED_BASIC_BUFFER_DEPTH	16 bytes each — most OpenLCB messages fit here
USER_DEFINED_DATAGRAM_BUFFER_DEPTH	72 bytes each — datagram protocol messages
USER_DEFINED_SNIP_BUFFER_DEPTH	256 bytes each — SNIP replies and payload events
USER_DEFINED_STREAM_BUFFER_DEPTH	USER_DEFINED_STREAM_BUFFER_LEN bytes each — stream data transfer

5.3 Node Count

USER_DEFINED_NODE_BUFFER_DEPTH controls how many virtual nodes this device can host. A simple accessory decoder uses 1. A command station managing multiple locomotives may use one slot per active locomotive.

5.4 Event Counts

```
#define USER_DEFINED_PRODUCER_COUNT    64 // max events this node can produce
#define USER_DEFINED_CONSUMER_COUNT    64 // max events this node can react to
#define USER_DEFINED_PRODUCER_RANGE_COUNT 5 // must be >= 1
#define USER_DEFINED_CONSUMER_RANGE_COUNT 5 // must be >= 1
```

5.5 Stream Transport

These constants configure the Stream Transport protocol. They are only meaningful when OPENLCB_COMPILE_STREAM is defined.

```

// Maximum bytes per stream data frame this node can accept.
// The spec uses a 2-byte field so the protocol max is 65535.
// During negotiation the smaller of the two nodes' buffer sizes wins.
#define USER_DEFINED_STREAM_BUFFER_LEN      256

// How many streams can be open at the same time across all nodes.
// Each active stream uses a small state struct, not a full payload buffer.
#define USER_DEFINED_MAX_CONCURRENT_ACTIVE_STREAMS  1

```

5.6 Train Protocol

These constants are only meaningful when OPENLCB_COMPILE_TRAIN is defined.

```

#define USER_DEFINED_TRAIN_NODE_COUNT      4    // max simultaneous train nodes
#define USER_DEFINED_MAX_LISTENERS_PER_TRAIN  6    // max consist members per train
#define USER_DEFINED_MAX_TRAIN_FUNCTIONS    29    // DCC functions: F0 through F28

```

5.7 Listener Alias Verification

Controls the periodic alias re-verification for consist listener entries. Only meaningful when OPENLCB_COMPILE_TRAIN is defined.

```

#define USER_DEFINED_LISTENER_PROBE_TICK_INTERVAL  1    // 100ms ticks between prober calls
#define USER_DEFINED_LISTENER_PROBE_INTERVAL_TICKS 250  // 100ms ticks between probes of same entry
#define USER_DEFINED_LISTENER_VERIFY_TIMEOUT_TICKS 30   // 100ms ticks to wait for AMD reply

```

5.8 DCC CV Programming Space

Only meaningful when OPENLCB_COMPILE_DCC_CV is defined. The library keeps a small table of CV requests that the application has not answered yet, so a slow decoder read never stalls the node.

```

#define USER_DEFINED_DCC_CV_PENDING_COUNT      4    // CV requests waiting on the track at once
#define USER_DEFINED_DCC_CV_TIMEOUT_TICKS      80    // 100ms ticks before an unanswered request fails
#define USER_DEFINED_DCC_CV_REPLY_TIME_SECONDS  4    // reply time advertised to the requester

```

6. Initialization — main.ino

The main sketch file wires together the hardware drivers, the library stack, and your application callbacks. The pattern is always the same four steps: define the config structs, initialize hardware, initialize the library, create your node(s).

6.1 The CAN Config Struct

This struct tells the CAN state machine how to talk to your CAN hardware. All fields are function pointers. The library calls these when it needs to send a frame, check the TX buffer, or notify you of incoming frames and alias changes.

```
static const can_config_t can_config = {
    .transmit_raw_can_frame = &Esp32CanDriver_transmit_raw_can_frame,
    .is_tx_buffer_clear     = &Esp32CanDriver_is_can_tx_buffer_clear,
    .lock_shared_resources  = &Esp32Drivers_lock_shared_resources,
    .unlock_shared_resources = &Esp32Drivers_unlock_shared_resources,
    .on_rx                  = &CallbacksCan_on_rx,          // optional
    .on_tx                  = &CallbacksCan_on_tx,          // optional
    .on_alias_change        = &CallbacksCan_on_alias_change, // optional
};
```

6.2 The OpenLCB Config Struct

This struct tells the OpenLCB protocol engine how to interact with your application. The most important entries are the config memory read/write functions and the reboot function. Fields are conditionally compiled based on your OPENLCB_COMPILE_* flags.

```
static const openlcb_config_t openlcb_config = {
    // REQUIRED: mutex for thread safety
    .lock_shared_resources  = &Esp32Drivers_lock_shared_resources,
    .unlock_shared_resources = &Esp32Drivers_unlock_shared_resources,

    // Memory Configuration (requires OPENLCB_COMPILE_MEMORY_CONFIGURATION)
    .config_mem_read        = &Esp32Drivers_config_mem_read,
    .config_mem_write       = &Esp32Drivers_config_mem_write,
    .reboot                 = &Esp32Drivers_reboot,          // optional
    .factory_reset          = &CallbacksConfigMem_factory_reset, // optional

    // Firmware Upgrade (requires OPENLCB_COMPILE_FIRMWARE)
    .freeze                 = &Drivers_freeze,
    .unfreeze               = &Drivers_unfreeze,
    .firmware_write         = &Drivers_firmware_write,

    // DCC CV programming (requires OPENLCB_COMPILE_DCC_CV)
    .dcc_cv_read            = &CallbacksDccCv_dcc_cv_read,    // optional
    .dcc_cv_write           = &CallbacksDccCv_dcc_cv_write,    // optional

    // Application callbacks
    .on_100ms_timer         = &CallbacksOlcb_on_100ms_timer,
    .on_consumed_event_pcer = &CallbacksEvents_on_consumed_event_pcer, // optional (EVENTS)
};
```

The struct contains additional optional callback fields for event identification, broadcast time, train control, train search, stream transport, and the config memory reply-time estimates (config_mem_read/write_delayed_reply_time, which return seconds). Set any field to NULL (or omit it) to use the library default. See openlcb_config.h for the full list.

6.3 Setup and Loop

```
void setup() {
    Serial.begin(9600);

    // 1. Initialize hardware
    Esp32CanDriver_initialize();
    Esp32Drivers_initialize();

    // 2. Initialize the library stacks
    CanConfig_initialize(&can_config);
    OpenLcbConfig_initialize(&openlcb_config);
}
```

```

// 3. Initialize your application callbacks
CallbacksOlc_initialize();

// 4. Create the node
OpenLcbConfig_create_node(NODE_ID, &OpenLcbUserConfig_node_parameters);
}

void loop() {
    // Run all state machines. Call as fast as possible.
    OpenLcbConfig_run();

    // Your application code here. Keep it short and non-blocking.
}

```

Note: `OpenLcbConfig_run()` must be called repeatedly from your main loop. Do not use `delay()` in `loop()` — it stalls the state machines.

7. Implementing the Drivers

The two driver files are the only hardware-specific code in a project. If you are using the ESP32 BasicNode example, these are already implemented. This section explains what each function must do when porting to a new platform.

7.1 CAN Bus Drivers (esp32_can_drivers.cpp)

transmit_raw_can_frame

Called by the library to send one CAN frame. The library will not call this again until `is_tx_buffer_clear()` returns true.

```
bool Esp32CanDriver_transmit_raw_can_frame(can_msg_t *can_msg) {
    twai_message_t msg = {};
    msg.extd = 1; // OpenLCB uses 29-bit extended IDs
    msg.identifier = can_msg->identifier;
    msg.data_length_code = can_msg->payload_count;
    for (int i = 0; i < can_msg->payload_count; i++)
        msg.data[i] = can_msg->payload[i];
    return (twai_transmit(&msg, 0) == ESP_OK); // non-blocking
}
```

is_tx_buffer_clear

```
bool Esp32CanDriver_is_can_tx_buffer_clear(void) {
    twai_status_info_t info;
    twai_get_status_info(&info);
    return ((TX_QUEUE_LEN - info.msgs_to_tx) > 0); // room for one more frame
}
```

Receiving Frames

Incoming CAN frames are not pulled by the library. Your setup code creates a FreeRTOS task (or polling loop) that reads frames from the hardware and passes them into the library:

```
void can_receive_task(void *param) {
    twai_message_t msg;
    for (;;) {
        if (twai_receive(&msg, pdMS_TO_TICKS(10)) == ESP_OK) {
            if (!msg.extd) continue; // OpenLCB frames are extended only
            can_msg_t can_msg;
            can_msg.identifier = msg.identifier;
            can_msg.payload_count = msg.data_length_code;
            for (int i = 0; i < msg.data_length_code; i++)
                can_msg.payload[i] = msg.data[i];
            CanRxStateMachine_incoming_can_driver_callback(&can_msg);
        }
    }
}
```

7.2 OpenLCB Platform Drivers (esp32_drivers.cpp)

Configuration Memory Read and Write

The library calls these when JMRI reads or writes your node's settings. You map addresses to your NVS or EEPROM.

```
uint16_t Esp32Drivers_config_mem_read(openlcb_node_t *node,
    uint32_t address, uint16_t count,
    configuration_memory_buffer_t *buffer)
{
    preferences.getBytes("cfg", scratch_buf, sizeof(scratch_buf));
    memcpy(buffer, scratch_buf + address, count);
    return count;
}

uint16_t Esp32Drivers_config_mem_write(openlcb_node_t *node,
    uint32_t address, uint16_t count,
    configuration_memory_buffer_t *buffer)
{
    preferences.getBytes("cfg", scratch_buf, sizeof(scratch_buf));
    memcpy(scratch_buf + address, buffer, count);
}
```

```
    preferences.putBytes("cfg", scratch_buf, sizeof(scratch_buf));  
    return count;  
}
```

8. Callbacks — Where Your Application Lives

Callbacks are function stubs the library calls when protocol events occur. This is where you write your application logic. The Node Wizard generates the function signatures with TODO comments. You fill in the bodies.

CallbacksOlc_b_initialize()

Called once during setup, after `OpenLcbConfig_initialize()`. Use this to register event IDs, load settings from NVS, and initialize output pins.

```
void CallbacksOlc_b_initialize(void) {
    openlcb_node_t *node = OpenLcbNode_get_first(0);

    // Register events this node produces
    OpenLcbApplication_register_producer_eventid(
        node, 0x0501010107770001, EVENT_STATUS_CLEAR);

    // Register events this node consumes
    OpenLcbApplication_register_consumer_eventid(
        node, 0x0501010107770002, EVENT_STATUS_UNKNOWN);

    pinMode(BUTTON_PIN, INPUT_PULLUP);
    pinMode(LED_PIN, OUTPUT);
}
```

CallbacksOlc_b_on_100ms_timer()

The library calls this function every 100 milliseconds. Good place for polling inputs and debouncing buttons.

```
static uint8_t button_prev = HIGH;

void CallbacksOlc_b_on_100ms_timer(void) {
    openlcb_node_t *node = OpenLcbNode_get_first(0);
    uint8_t button_now = digitalRead(BUTTON_PIN);
    if (button_now == LOW && button_prev == HIGH)
        OpenLcbApplication_send_event_pc_report(
            node, 0x0501010107770001);
    button_prev = button_now;
}
```

CAN Observation Callbacks (Optional)

`CallbacksCan_on_rx` and `CallbacksCan_on_tx` let you observe every raw CAN frame. Most applications leave these as empty stubs. Useful for debugging.

9. Events in Depth

Choosing Event IDs

An Event ID is a 64-bit number. The convention is: the upper 6 bytes come from your Node ID, and the lower 2 bytes are a sequence number you choose. This ensures your event IDs are globally unique.

```
// Pattern: Node ID bytes + your own sequence number
// Node ID: 0x05.01.01.01.07.77
#define EVENT_BUTTON_PRESSED    0x0501010107770001
#define EVENT_BUTTON_RELEASED   0x0501010107770002
#define EVENT_SENSOR_ACTIVE     0x0501010107770003
#define EVENT_SENSOR_INACTIVE   0x0501010107770004
```

Event Status Values

Status	Meaning
EVENT_STATUS_CLEAR	The condition this event represents is currently inactive/off.
EVENT_STATUS_SET	The condition is currently active/on.
EVENT_STATUS_UNKNOWN	Current state is unknown (most common for consumers at startup).

Note: Event IDs in configuration memory are big-endian (most significant byte first). On little-endian processors like ESP32, you may need to byte-swap them after reading.

10. Configuration Memory

When OPENLCB_COMPILE_MEMORY_CONFIGURATION is enabled, configuration tools like JMRI can read and write your node's settings remotely. The settings are stored as a flat byte array in non-volatile storage. The CDI tells JMRI how to interpret the bytes.

Memory Space Layout

Space	Purpose
255 (0xFF)	CDI XML. The library serves the CDI byte array from here (read only).
254 (0xFE)	All memory (read only). Raw view of the node's memory.
253 (0xFD)	User configuration. The CDI describes this space; JMRI reads and writes here through your config_mem_read/write drivers.
252 (0xFC)	ACDI manufacturer information (read only), served from the SNIP data.
251 (0xFB)	ACDI user information: user name and description, read/write.
250 (0xFA)	FDI XML (train nodes only, read only).
249 (0xF9)	Train function configuration memory (train nodes only).
248 (0xF8)	DCC CV programming: one CV per byte, address = CV number - 1 (OPENLCB_COMPILE_DCC_CV).
239 (0xEF)	Firmware upgrade image (OPENLCB_COMPILE_FIRMWARE).

DCC CV Programming (Space 0xF8)

With OPENLCB_COMPILE_DCC_CV enabled, JMRI DecoderPro and OpenMRN tools read and write decoder CVs through this node. Service-mode (programming track) requests go to the command station node; ops-mode (POM) requests go to the train node. Your dcc_cv_read and dcc_cv_write callbacks may answer at once, or return OPENLCB_DCC_CV_RESULT_PENDING and report the result later with OpenLcbApplicationDccCv_complete(handle, result, value) from the main loop, so waiting for a decoder acknowledgement or a RailCom cutout never stalls the node. The library advertises a reply time of USER_DEFINED_DCC_CV_REPLY_TIME_SECONDS, absorbs a requester's re-send of an outstanding request, and answers with a temporary time-out if the application never completes it.

11. Train Nodes (Brief Overview)

Select Train or Train Controller in the Node Wizard to add train-specific protocols.

Train Node

A Train node represents a single locomotive and accepts speed and function commands from throttles over the LCC network. The `openlcb_config_t` struct contains callback fields for speed changes, function changes, emergency stop, controller assignment, and listener management. All are optional — set to NULL to use library defaults.

Train Controller

A Train Controller (throttle or command station) discovers and commands trains on the network using the Train Search protocol. This node type does not have an FDI.

12. Using PlatformIO Instead of Arduino IDE

PlatformIO in VSCode is an alternative to Arduino IDE. Here is a minimal `platformio.ini` for the BasicNode example:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 921600
build_flags = -std=gnu99
build_src_filter = -<*>
                  +<*.c> +<*.cpp> +<*.h>
                  +<openlcb_c_lib/openlcb/*.c> +<openlcb_c_lib/openlcb/*.h>
                  +<openlcb_c_lib/drivers/canbus/*.c> +<openlcb_c_lib/drivers/canbus/*.h>
                  +<openlcb_c_lib/utilities/*.c> +<openlcb_c_lib/utilities/*.h>
                  +<application_callbacks/*.c> +<application_callbacks/*.h>
                  +<application_drivers/*.c> +<application_drivers/*.cpp>
```

The source filter lists exactly which folders under `src/` are compiled, which keeps the unused transport driver out of the build. The full file is in `applications/platformio/esp32/BasicNode/`.

13. Troubleshooting

Symptom	Likely Cause and Fix
Node does not appear in JMRI	Check CAN wiring (TX/RX may be swapped). Verify 120-ohm termination resistors on bus. Check <code>NODE_ID</code> is unique.
Undefined reference to <code>USER_DEFINED_*</code>	<code>openlcb_user_config.h</code> is not on the include path. For Arduino IDE it must be in the sketch folder. For PlatformIO add <code>-I src</code> .
Node appears then immediately disappears	Another node has the same Node ID. Every node must have a unique ID.
JMRI config panel is empty	CDI is malformed or the CDI byte array was not regenerated after editing the XML. Re-run <code>tools/xml_to_array</code> or regenerate the project with the Node Wizard.
Settings do not save across power cycles	<code>config_mem_write</code> is not writing to NVS. Add <code>Serial.println()</code> in the write function to confirm it is being called.
Events not received by consumer	Verify both nodes are on the same bus. Verify the consumer registered the exact same 64-bit event ID as the producer.
Buffer depth compile error on 8-bit processors	Total buffer count (sum of all four depth defines) must not exceed 126 on 8-bit platforms.

Enabling Verbose Compilation Output

```
// In openlcb_user_config.h:
#define OPENLCB_COMPILE_VERBOSE
```

```
// Prints a feature summary confirming which protocols were compiled in.
```

Online Resources

Library documentation: <https://jimkueneman.github.io/OpenLcbCLib/>

OpenLCB/LCC standards: <https://www.nmra.org/lcc>